Assignment Submission

Course Code: CSE 323

Course Name: Operating Systems

Topic Name: Assignment-1

Submitted To

Name: Nushrat Jahan Oyshi

Designation: Lecturer

Department: CSE

Daffodil International University

Submitted By

Name: Md Taslimun Hasan

ID: 232-15-688

Section: 65_F

Semester: Spring-2026

Department: CSE

Daffodil International University

Submission Date: 21/02/2026

www.diucoverpage.com

Summer-2025

1. a) Batch vs Real-time vs Distributed OS

- Batch OS: Jobs are collected and executed without user interaction.

  Performance: Efficient for large jobs, but poor response time.

  Resource Management: Sequential allocation.

  Scheduling: Non-preemptive, first-come-first-serve.
- Real-time OS: Designed for tasks with strict timing constraints.

  Performance: Fast response, predictable.

  Resource Management: Prioritizes critical tasks.

  Scheduling: Preemptive, priority-based.
- Distributed OS: Manages multiple machines as one system.

  Performance: High scalability, fault tolerance.

  Resource Management: Shared across nodes.

  Scheduling: Load balancing across systems.

2. a) USB device access involves:

- Device Driver: Handles communication with hardware.
- I/O Subsystem: Manages data transfer between device and memory.

3. a) Lightweight OS for IoT

Suitable Architecture:

- Microkernel architecture (minimal kernel, modular services).

Justification:

- Modularity: Easy to add or remove services.
- Performance: Lightweight, less overhead.
- Reliability: Fault isolation. A failed service doesn’t crash the kernel.

4. **C Program**

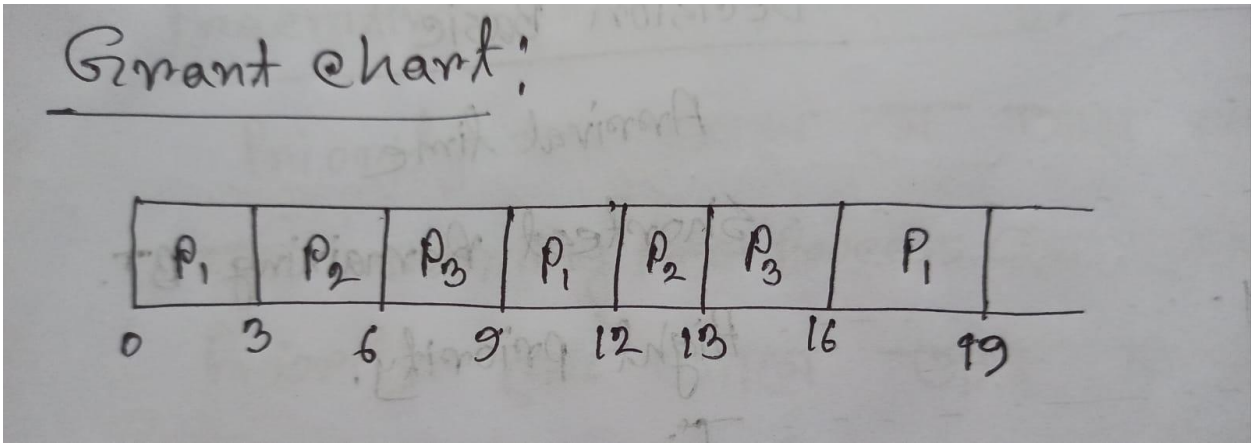
```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>

int main() {

    pid_t pid = fork();


    if (pid < 0) {

        perror("Fork failed");

        exit(1);

    } else if (pid == 0) { // Child process

        int fd = open("test.txt", O_RDONLY);

        if (fd < 0) {

            perror("File open failed");

            exit(1);

        }

        char buffer[100];

        read(fd, buffer, sizeof(buffer));

        printf("Child read: %s\n", buffer);

        close(fd);

        exit(0);

    } else { // Parent process

        wait(NULL);

        printf("Parent: Child finished execution.\n");

    }

    return 0;

}
```

4. Round Robin Scheduling (Quantum = 3 ms)

Processes:

- P1: Arrival 0, Burst 10
- P2: Arrival 1, Burst 4
- P3: Arrival 2, Burst 6

i). Gantt Chart:



Turnaround Time (TAT):

- P1: 19 - 0 = 19
- P2: 13 - 1 = 12
- P3: 16 - 2 = 14

Waiting Time (WT):

- P1: (TAT 19 - Burst 10) = 9
- P2: (TAT 12 - Burst 4) = 8
- P3: (TAT 14 - Burst 6) = 8

ii). Fairness:

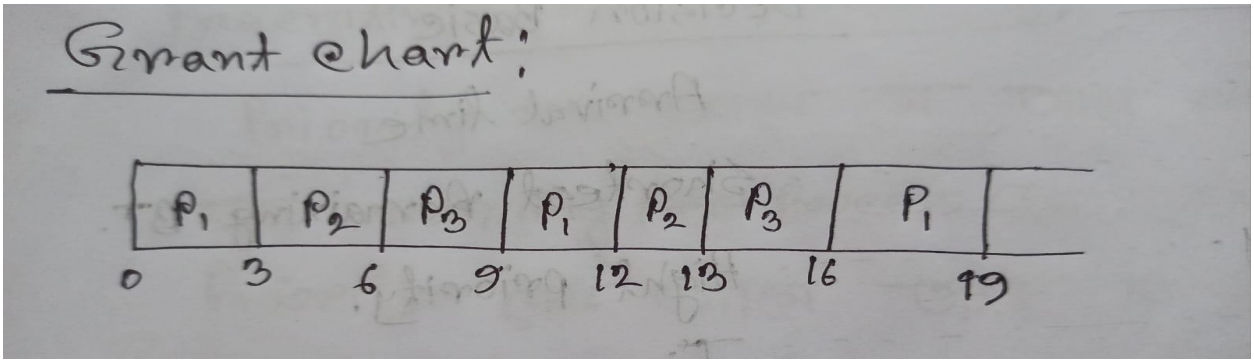
- Each process gets equal CPU slices.
- Drawback: Context switching overhead may reduce responsiveness in real-time systems.

5. Preemptive Priority Scheduling

Processes:

- P1: Arrival 0, Burst 7, Priority 2
- P2: Arrival 1, Burst 4, Priority 1
- P3: Arrival 2, Burst 6, Priority 3
- P4: Arrival 3, Burst 5, Priority 2

i). Grant chart:



Turnaround Time (TAT):

- P1: 10 - 0 = 10
- P2: 5 - 1 = 4
- P3: 21 - 2 = 19
- P4: 15 - 3 = 12

Waiting Time (WT):

- P1: (Finish 10 - Arrival 0 - Burst 7) = 3
- P2: (Finish 5 - Arrival 1 - Burst 4) = 0
- P3: (Finish 21 - Arrival 2 - Burst 6) = 13
- P4: (Finish 15 - Arrival 3 - Burst 5) = 7

ii). Comparison:

- Preemptive: Responds quickly to high-priority tasks, but low-priority may starve.
- Non-preemptive: Fairer for long tasks, but slower response for urgent tasks.

Fall-2025

1(a)

Mr. X connects his PC to the printer via a **printer server**. The printer server acts as a **shared resource manager**, allowing multiple PCs to send print jobs. The OS uses **spooling**: jobs are queued in a buffer, then sent to the printer sequentially. PC → Printer Driver → Printer Server → Printer. This ensures efficient file sharing and prevents conflicts when multiple users print simultaneously.

1(b) Type of OS for Printer Server

Spooling-based OS (Batch-like system) is used. It queues jobs, ensures fairness, and allows multiple users to share the printer without blocking each other.

2(a) Chrome Handling Resources

- Chrome uses a **multi-process architecture**: each tab, extension, and plugin runs in a separate process.
- **Memory Management**: Each process has an isolated memory space, preventing crashes from spreading.
- **Communication Model**: Processes communicate via IPC.
- **Efficiency & Security**: Isolation improves stability, and sandboxing protects against malicious websites.

2(b) API, System Calls, and OS

API: Provides functions for developers (e.g., open(), read()).

System Call: Actual request to the OS kernel (e.g., sys_open).

OS: Executes the system call, manages hardware, and returns results.

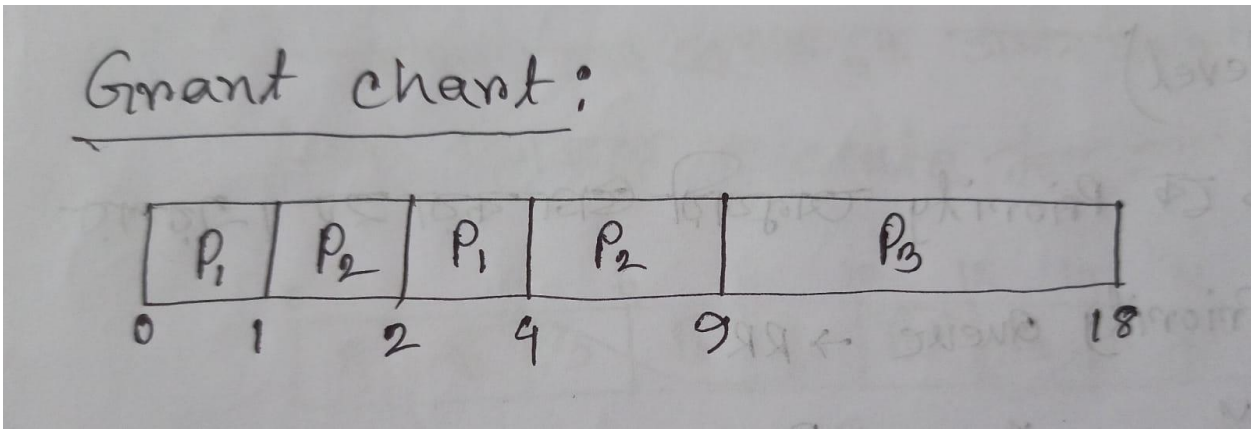
Relationship:

Application → API → System Call → OS Kernel → Hardware

Example: When Chrome launches, the OS creates a process via a system call, such as fork() or CreateProcess.

3(a)(i) Processes for Scheduling

Gantt Chart



Completion Times (CT):

- P1 = 4
- P2 = 9
- P3 = 18

Turnaround Time (TAT = CT – AT):

- P1 = 4 – 0 = 4
- P2 = 9 – 1 = 8
- P3 = 18 – 2 = 16

Waiting Time (WT = TAT – BT):

- P1 = 4 – 4 = 0
- P2 = 8 – 5 = 3
- P3 = 16 – 9 = 7

Averages:

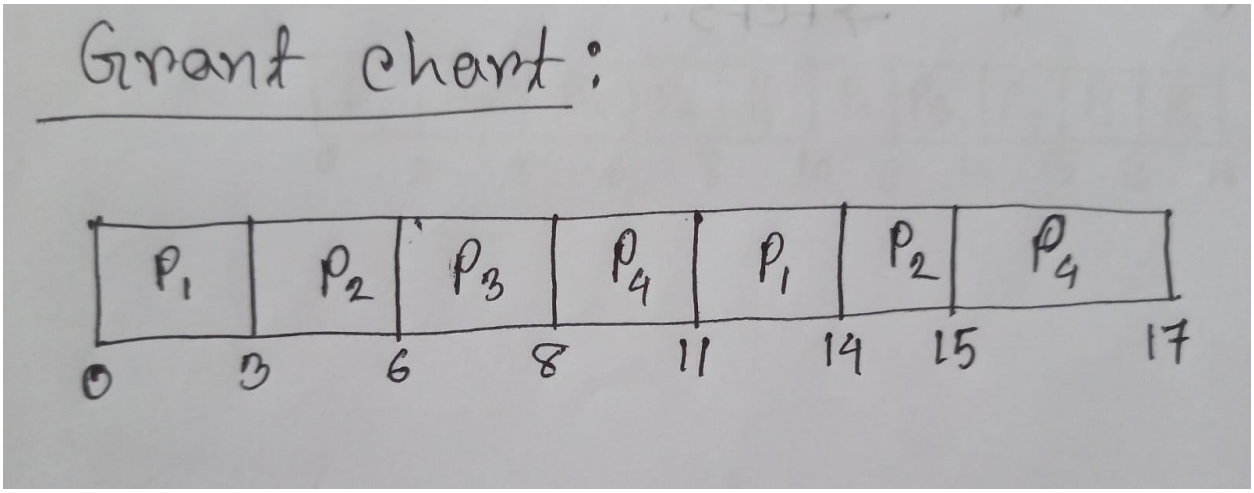
- Avg TAT = (4+8+16)/3 = **9.33**
- Avg WT = (0+3+7)/3 = **3.33**

3(a) (ii) Advantages of SRTF

- Minimizes average waiting time.
- Efficient for short processes.
- Ensures faster response for quick tasks.
- Considered efficient because it always executes the shortest remaining job first.

3(b) (i) Round Robin (Quantum = 3)

Gantt Chart (q=3)



Completion Times (CT):

- P1 = 14
- P2 = 15
- P3 = 8
- P4 = 17

Turnaround Time (TAT = CT – AT):

- P1 = 14 – 0 = 14
- P2 = 15 – 1 = 14
- P3 = 8 – 2 = 6
- P4 = 17 – 3 = 14

Waiting Time (WT = TAT – BT):

- P1 = 14 – 6 = 8
- P2 = 14 – 4 = 10
- P3 = 6 – 2 = 4
- P4 = 14 – 5 = 9

Averages:

- Avg TAT = (14+14+6+14)/4 = **12**
- Avg WT = (8+10+4+9)/4 = **7.75**

3(b) Effect of Quantum Length

- **Small Quantum**: More context switching, higher overhead, better responsiveness.
- **Large Quantum**: Less context switching, but behaves like FCFS (poor responsiveness).
- **Balanced Quantum**: Ensures fairness and good performance.